

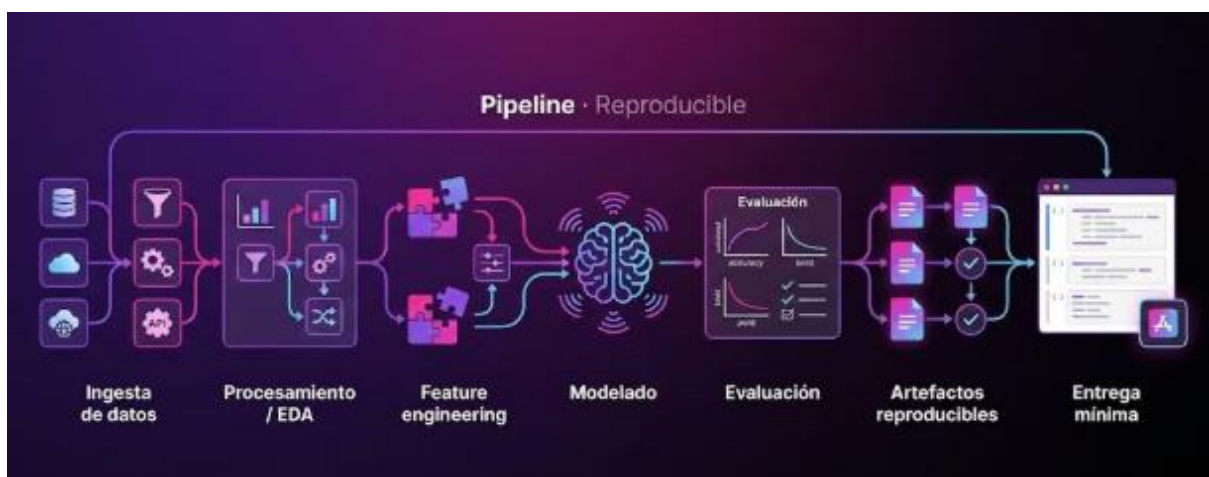
1 - Principios de diseño de pipelines reproducibles

En esta unidad, exploraremos los fundamentos para diseñar pipelines de ciencia de datos que sean reproducibles y eficientes. Aprenderás la estructura completa de un pipeline end-to-end, la gestión adecuada de artefactos y la importancia del control de versiones para garantizar que tus resultados sean confiables y compartibles. Además, conocerás prácticas para el despliegue mínimo que facilitan la entrega de soluciones funcionales en entornos reales.

¿Qué hace que un pipeline de ciencia de datos sea realmente reproducible?

Imagina que has desarrollado un modelo predictivo que mejora significativamente la toma de decisiones en tu empresa. Sin embargo, cuando otro colega intenta replicar tu trabajo, los resultados no coinciden. ¿Qué salió mal? La reproducibilidad es clave para que los proyectos de ciencia de datos sean confiables y escalables en la industria.

En esta unidad, descubrirás cómo diseñar pipelines end-to-end que no solo funcionen, sino que puedan ser replicados y compartidos fácilmente. Exploraremos desde la estructura básica de un pipeline, pasando por la gestión de artefactos y control de versiones, hasta las mejores prácticas para un despliegue mínimo que facilite la entrega de tus soluciones.



1. ¿Qué es un pipeline end-to-end en ciencia de datos?

Un *pipeline end-to-end* es un flujo completo que abarca todas las etapas necesarias para transformar datos crudos en un modelo funcional y evaluado, listo para su uso o despliegue. Este flujo incluye:

- **Ingestión de datos:** Recolección y carga de datos desde diversas fuentes.
- **Análisis exploratorio de datos (EDA):** Comprensión inicial de los datos, detección de patrones y limpieza.
- **Feature engineering:** Creación y selección de variables relevantes para el modelo.
- **Modelado:** Entrenamiento de algoritmos para aprender patrones.
- **Evaluación:** Medición del desempeño del modelo con métricas adecuadas.
- **Entrega mínima:** Preparación para compartir o desplegar el modelo, por ejemplo, mediante notebooks reproducibles o servicios simples.

Nota: Este pipeline debe ser diseñado para que cualquier persona pueda seguirlo y obtener resultados consistentes.

2. Componentes clave para la reproducibilidad

Para que un pipeline sea reproducible, es fundamental considerar:

- **Gestión de artefactos:** Guardar versiones de datasets, modelos entrenados, y resultados intermedios.
- **Control de versiones:** Uso de sistemas como Git para rastrear cambios en código y documentación.
- **Entornos reproducibles:** Definir dependencias y versiones de librerías para evitar discrepancias.
- **Documentación clara:** Explicar cada paso y decisión tomada.

3. Prácticas para despliegue mínimo y compartición

El despliegue mínimo busca entregar una versión funcional del pipeline que permita a otros reproducir y validar resultados sin complejidades innecesarias. Algunas prácticas incluyen:

- Uso de **notebooks** bien estructurados (Jupyter, Google Colab) que integren código, visualizaciones y explicaciones.
- Repositorios organizados con código, datos y documentación.
- Guardado de artefactos con nombres y formatos estándar.
- Creación de demos simples con herramientas como Streamlit o Flask para mostrar resultados interactivos.

Estas prácticas facilitan la colaboración y aceleran la adopción de soluciones en entornos empresariales.

En la industria, los pipelines reproducibles son esenciales para garantizar que los modelos de ciencia de datos puedan ser auditados, mejorados y desplegados sin sorpresas. Por ejemplo, en un proyecto de detección de fraude, un pipeline reproducible permite que el equipo de ingeniería valide el modelo antes de integrarlo en producción, asegurando que los resultados sean consistentes y confiables.

Además, la gestión adecuada de artefactos y el control de versiones evitan pérdidas de trabajo y facilitan la colaboración entre equipos multidisciplinarios. El despliegue mínimo, por su parte, permite entregar prototipos funcionales que pueden ser evaluados por stakeholders sin necesidad de infraestructuras complejas.

Esta unidad sienta las bases conceptuales para que, en las siguientes unidades, puedas implementar y experimentar con pipelines reproducibles usando notebooks y herramientas prácticas.

2 - Casos de estudio: segmentación y recomendaciones

Análisis comparativo de métodos, requisitos y ejemplos de métricas de negocio para segmentación de clientes y sistemas de recomendación en la industria.

Introducción

Imagina que trabajas en una empresa de comercio electrónico que quiere aumentar sus ventas personalizando la experiencia de sus clientes. ¿Cómo identificar grupos de clientes con comportamientos similares para ofrecer promociones específicas? ¿O cómo recomendar productos que realmente interesen a cada usuario? Estas preguntas son el corazón de dos aplicaciones clave del Machine Learning (ML) en la industria: la **segmentación de clientes** y los **sistemas de recomendación**.

En esta unidad, reforzaremos el conocimiento sobre casos de uso reales de ML, enfocándonos en estos dos ejemplos prácticos. Exploraremos los métodos más comunes, los requisitos de datos que necesitan y las métricas que permiten evaluar su éxito en un contexto comercial. Al finalizar, podrás comparar diferentes enfoques y entender sus ventajas y limitaciones, preparándote para diseñar soluciones efectivas en proyectos reales de ciencia de datos.

Casos de uso de ML en la industria: Segmentación y Recomendaciones

Segmentación de Clientes

La segmentación consiste en dividir una población de clientes en grupos homogéneos según características o comportamientos similares. Esto permite personalizar estrategias de marketing, mejorar la retención y optimizar recursos.

- **Métodos comunes:**
- *Clustering no supervisado:* algoritmos como K-means, DBSCAN o clustering jerárquico que agrupan sin etiquetas previas.
- *Segmentación basada en reglas:* usando criterios definidos por expertos.
- **Requisitos de datos:**
- Variables relevantes y limpias (demográficas, transaccionales, comportamiento web).
- Suficiente volumen para detectar patrones significativos.
- **Métricas de éxito:**
- *Silhouette Score:* mide la cohesión y separación de clusters.
- Impacto en KPIs comerciales: aumento en ventas, tasa de retención, conversión.

Sistemas de Recomendación

Los sistemas de recomendación sugieren productos o contenidos personalizados para cada usuario, aumentando la satisfacción y ventas.

- **Tipos principales:**
- *Filtrado colaborativo:* recomienda basado en similitud entre usuarios o ítems.
- *Filtrado basado en contenido:* usa características del producto y preferencias del usuario.
- *Modelos híbridos:* combinan ambos enfoques para mejorar precisión.
- **Requisitos de datos:**
- Historial de interacciones usuario-producto.
- Información contextual (tiempo, ubicación).
- **Métricas de éxito:**
- *Precisión y recall:* qué tan acertadas son las recomendaciones.

- *Tasa de clics (CTR) y conversión.*
- *Diversidad y novedad para evitar recomendaciones repetitivas.*

***Nota:** Ambos casos requieren un entendimiento profundo del negocio para seleccionar variables y métricas adecuadas.*

Comparación y Trade-offs

Aspecto	Segmentación	Recomendaciones
Tipo de ML	No supervisado	Supervisado / híbrido
Objetivo	Agrupar clientes	Personalizar experiencia
Datos requeridos	Variables descriptivas	Interacciones usuario-producto
Métricas clave	Cohesión de grupos, impacto negocio	Precisión, CTR, diversidad
Complejidad	Moderada	Alta (requiere modelado avanzado)

Esta tabla sintetiza las diferencias clave para ayudarte a elegir el enfoque según el problema y recursos disponibles.

Aplicación práctica en la industria

En la práctica, una empresa puede usar segmentación para identificar grupos de clientes con alta propensión a comprar un nuevo producto y luego aplicar sistemas de recomendación para personalizar ofertas dentro de cada segmento.

Por ejemplo, un retailer online puede segmentar a sus clientes en base a frecuencia de compra y categorías preferidas, y luego usar un sistema de recomendación híbrido para sugerir productos nuevos o complementarios.

Este enfoque combinado maximiza el impacto comercial y optimiza el uso de datos y recursos.

Además, entender las métricas de negocio y técnicas de evaluación es crucial para medir el éxito y ajustar las estrategias en ciclos iterativos.

En la siguiente unidad, profundizaremos en la detección de fraude y optimización logística, ampliando el panorama de aplicaciones de ML en la industria.

Casos de Éxito

San Cristóbal – Detección de Fraudes

Introducción

¿Cómo puede una empresa como San Cristóbal protegerse eficazmente contra fraudes que amenazan su operación y confianza con los clientes? En este módulo, abordaremos un caso real donde se implementa un pipeline de detección de fraudes basado en modelos supervisados, complementado con técnicas avanzadas de Deep Learning para el análisis de imágenes de siniestros. A lo largo de esta unidad, descubrirás cómo seleccionar métricas clave como precisión, recall y AUC para evaluar modelos en contextos con datos desbalanceados, y cómo integrar análisis visuales para fortalecer la investigación de fraudes. Este conocimiento te permitirá comprender no solo la teoría, sino también la aplicación práctica en un entorno empresarial real, preparando el terreno para replicar y adaptar estas soluciones en tus propios proyectos de Data Science.

Teoría

Pipeline de Detección de Fraudes en San Cristóbal

La **detección de fraude** es un problema clásico de clasificación supervisada donde el objetivo es identificar transacciones o eventos fraudulentos entre un gran volumen de datos legítimos. En San Cristóbal, el pipeline se compone de varias etapas clave:

1. **Detección inicial:** Uso de modelos supervisados entrenados con datos históricos para clasificar eventos como fraudulentos o no.
2. **Investigación:** Análisis detallado, que incluye revisión manual y análisis de imágenes de siniestros mediante Deep Learning.
3. **Resolución:** Toma de decisiones basadas en resultados y métricas para mitigar el fraude.

Modelos Supervisados y Balanceo de Clases

Dado que los fraudes suelen ser eventos raros, los datos están altamente desbalanceados. Para abordar esto, se aplican técnicas como:

- **Oversampling:** Aumentar artificialmente la cantidad de ejemplos minoritarios (fraudes) para equilibrar el dataset.
- **Undersampling:** Reducir la cantidad de ejemplos mayoritarios para balancear las clases.

Estas técnicas mejoran la capacidad del modelo para detectar fraudes sin sesgarse hacia la clase mayoritaria.

Métricas de Evaluación y Validación

En problemas de fraude, la **precisión** y el **recall** son métricas críticas:

- **Precisión (Precision):** Proporción de predicciones positivas que son correctas.
- **Recall (Sensibilidad):** Proporción de fraudes reales que el modelo detecta.

Además, el **AUC (Área bajo la curva ROC)** mide la capacidad del modelo para distinguir entre clases en diferentes umbrales.

Se utilizan técnicas de validación como **cross-validation** para asegurar la robustez del modelo.

Aplicaciones de Deep Learning en Visión

Para complementar la detección basada en datos tabulares, San Cristóbal integra modelos de Deep Learning para analizar imágenes de siniestros. Las redes neuronales convolucionales (CNN) permiten extraer características automáticas y detectar patrones visuales indicativos de fraude.

***Nota:** Aunque no profundizaremos en la arquitectura de estas redes, es importante reconocer su rol en enriquecer el pipeline de detección.*

Visualización con Matplotlib

La visualización de resultados y métricas es fundamental para interpretar el desempeño del modelo y comunicar hallazgos a stakeholders. Matplotlib es la herramienta principal para crear gráficos claros y efectivos.

Contexto

En la práctica, San Cristóbal enfrenta el desafío de detectar fraudes en tiempo real para minimizar pérdidas y mejorar la confianza del cliente. El pipeline implementado combina modelos supervisados entrenados con datos históricos, técnicas para manejar el desbalance de clases y métricas específicas que priorizan la detección efectiva de fraudes.

Además, la integración de Deep Learning para el análisis de imágenes de siniestros permite identificar patrones visuales sospechosos que complementan la información tabular, enriqueciendo la investigación y validación de casos.

Este enfoque integral requiere un entendimiento claro de cada etapa, desde la selección de métricas hasta la interpretación de resultados, para garantizar que las soluciones sean efectivas y aplicables en un entorno empresarial real.

En la siguiente sección, te guiaremos para replicar este análisis en un Jupyter Notebook, utilizando Python, scikit-learn, pandas y matplotlib, consolidando así tu aprendizaje con una experiencia práctica.

Práctica

Práctica: Replicando el Pipeline de Detección de Fraudes de San Cristóbal

1. csv/creditcardfraud
2. **Carga y exploración de datos:** Importa el dataset de transacciones y realiza un análisis exploratorio para identificar el desbalance de clases.
3. **Limpieza y preprocesamiento:** Aplica técnicas de oversampling (SMOTE) o undersampling para equilibrar las clases.
4. **Entrenamiento de modelo supervisado:** Utiliza un clasificador como Random Forest o Logistic Regression para entrenar el modelo.
5. **Evaluación del modelo:** Calcula métricas de precisión, recall, F1 y AUC. Visualiza la matriz de confusión y la curva ROC con matplotlib.
6. **Interpretación y reporte:** Resume los hallazgos y justifica la selección de métricas y técnicas aplicadas.

Esta práctica te permitirá aplicar los conceptos aprendidos y entender cómo se combinan diferentes técnicas para abordar un problema real de detección de fraudes.

Código inicial:

Importaciones específicas

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, StratifiedKFold,
GridSearchCV
```



```

from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (precision_score, recall_score, f1_score,
                             roc_auc_score, confusion_matrix, roc_curve,
                             classification_report)
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

```

Exploración inicial orientada al desbalance

```

if df_fraud.empty:
    print("df_fraud está vacío.")
else:
    counts = df_fraud['Class'].value_counts()
    print("Conteo:\n", counts)
    print("Proporciones:\n", (counts / counts.sum()).round(4))
    plt.figure(figsize=(6,4))
    sns.barplot(x=counts.index, y=counts.values)
    plt.title("Distribución de clases")
    plt.show()
    vars_of_interest = [c for c in ['Time', 'Amount'] if c in df_fraud.columns]
    if vars_of_interest:
        df_fraud[vars_of_interest].hist(bins=40, figsize=(12,4))
        plt.suptitle("Histogramas: Time y Amount")
        plt.show()
    sample_corr = df_fraud.sample(n=min(20000, len(df_fraud)),
    random_state=RANDOM_STATE)
    corr = sample_corr.corr()
    plt.figure(figsize=(10,8))
    sns.heatmap(corr, cmap='coolwarm', center=0)
    plt.title("Matriz de correlación")
    plt.show()

```

Split estratificado (train/test)

```

target_col = 'Class'
exclude_cols = [target_col]
feature_cols = [c for c in df_fraud.columns if c not in exclude_cols]
X = df_fraud[feature_cols].copy()
y = df_fraud[target_col].copy()
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=RANDOM_STATE)
print("X_train:", X_train.shape, "X_test:", X_test.shape)

```

```

# Aplicación de SMOTE en entrenamiento
smote = SMOTE(random_state=RANDOM_STATE)
rf = RandomForestClassifier(n_estimators=100,
                           random_state=RANDOM_STATE,
                           n_jobs=-1)
pipeline_smote_rf = ImbPipeline(steps=[
    ('smote', smote),
    ('rf', rf)])
print("Entrenando modelo...")
pipeline_smote_rf.fit(X_train, y_train)
print("Entrenamiento finalizado.")

```

Medplaya – Analítica Predictiva en Hotelería

Introducción

Introducción

Imagina que eres parte del equipo de análisis de una cadena hotelera que enfrenta un desafío común pero crítico: las cancelaciones de reservas afectan la ocupación y los ingresos. ¿Cómo anticipar estas cancelaciones para optimizar la gestión de habitaciones y maximizar el revenue? En esta unidad, exploraremos el caso premiado de Medplaya, donde se aplicaron modelos de clasificación supervisada para predecir cancelaciones y diseñar estrategias efectivas como el overbooking controlado.

A lo largo de esta sesión, aprenderás a identificar y seleccionar las características más relevantes basadas en el comportamiento histórico y señales contextuales, construir pipelines reproducibles en Python y evaluar el impacto de las intervenciones en la ocupación hotelera. Este conocimiento te permitirá aplicar analítica predictiva en contextos reales de hotelería, mejorando la toma de decisiones y resultados de negocio.

Teoría

Analítica Predictiva en Hotelería: Predicción de Cancelaciones

La **analítica predictiva en hotelería** se enfoca en anticipar comportamientos futuros, como cancelaciones de reservas, para optimizar la ocupación y maximizar ingresos. Las cancelaciones afectan la planificación y pueden generar pérdidas significativas si no se gestionan adecuadamente.

Modelos de Clasificación Supervisada para Cancelaciones

Los modelos de clasificación supervisada predicen una etiqueta discreta, en este caso, si una reserva será cancelada (Sí o No). Entre los algoritmos más usados se encuentran:

- **Árboles de decisión:** Modelos interpretables que segmentan el espacio de características.
- **Random Forest:** Ensamble de árboles que mejora la precisión y reduce sobreajuste.
- **Regresión logística:** Modelo probabilístico para clasificación binaria.
- **Modelos de boosting** (e.g., XGBoost): Potencian modelos débiles para mejorar rendimiento.

***Nota:** La selección del modelo depende de la calidad de datos, interpretabilidad requerida y recursos computacionales.*

Selección y Construcción de Features

La calidad de las predicciones depende en gran medida de las características (features) usadas. En hotelería, se consideran:

- **Comportamiento histórico:** frecuencia de cancelaciones previas del cliente, tiempo entre reserva y fecha de llegada.
- **Señales contextuales:** temporada del año, eventos locales, tipo de habitación, canal de reserva.

La ingeniería de características (feature engineering) incluye:

- Transformaciones numéricas (e.g., días hasta la reserva)
- Variables categóricas codificadas (one-hot encoding)
- Creación de variables derivadas (e.g., tasa de cancelación por cliente)

Manejo del Desequilibrio en Datos

Las cancelaciones suelen ser menos frecuentes que las reservas confirmadas, generando un conjunto de datos desequilibrado. Técnicas para abordar esto incluyen:

- **Re-muestreo:** oversampling de la clase minoritaria o undersampling de la mayoritaria.
- **Algoritmos adaptados:** modelos que ponderan clases o usan métricas específicas.

Métricas de Evaluación

Para evaluar modelos de clasificación en este contexto, se usan:

Métrica	Descripción	Importancia en cancelaciones
Precisión	Proporción de predicciones correctas	Evita falsas alarmas de cancelación
Recall (Sensibilidad)	Proporción de cancelaciones correctamente detectadas	Crucial para anticipar cancelaciones reales
F1-Score	Balance entre precisión y recall	Útil en desequilibrio de clases
AUC-ROC	Capacidad de distinguir entre clases	Evalúa rendimiento global del modelo

Ejercicio de reflexión: ¿Por qué podría ser más importante maximizar el recall que la precisión en este caso?

Impacto en Negocio: Overbooking Controlado

Con predicciones confiables, se puede implementar **overbooking controlado**, aceptando más reservas que la capacidad real para compensar cancelaciones esperadas, optimizando la ocupación y el revenue.

Sin embargo, requiere un balance cuidadoso para evitar sobreventa y mala experiencia al cliente.

Contexto

Contexto Práctico y Aplicación en Medplaya

Medplaya, una cadena hotelera reconocida, enfrentaba pérdidas por cancelaciones imprevistas. Aplicaron un pipeline de analítica predictiva que incluyó:

1. **Recolección y limpieza de datos:** registros históricos de reservas, cancelaciones, características de clientes y contexto.
2. **Ingeniería de características:** extracción de variables clave como días hasta la llegada, historial de cancelaciones, tipo de reserva y eventos locales.
3. **Modelado supervisado:** entrenamiento de modelos como Random Forest y regresión logística para predecir cancelaciones.
4. **Evaluación rigurosa:** uso de métricas como F1-Score y AUC-ROC para seleccionar el mejor modelo.
5. **Implementación de estrategias:** diseño de políticas de overbooking basadas en predicciones para maximizar ocupación sin afectar la experiencia.

Este enfoque permitió a Medplaya aumentar la ocupación promedio y mejorar el revenue, demostrando el valor tangible de la analítica predictiva.

Amazon – Sistemas de Recomendación

Introducción

Introducción

Imagina que estás navegando en Amazon y recibes recomendaciones personalizadas que parecen anticipar exactamente lo que necesitas. ¿Cómo logra Amazon ofrecer estas sugerencias precisas entre millones de productos y usuarios? En esta unidad, analizaremos los sistemas de recomendación que sustentan estas experiencias, explorando desde métodos clásicos como la factorización de matrices hasta enfoques

híbridos que combinan múltiples técnicas. Además, entenderemos cómo estas soluciones impactan directamente en las ventas y la satisfacción del cliente, y qué desafíos implica su despliegue en entornos productivos en tiempo real.

Al finalizar, serás capaz de analizar críticamente diferentes algoritmos de recomendación, explicar sus fundamentos técnicos y evaluar su impacto en el negocio, preparándote para implementar y optimizar sistemas similares en contextos reales de Data Science.

Teoría

Fundamentos de Sistemas de Recomendación

Los sistemas de recomendación son herramientas clave en comercio electrónico para personalizar la experiencia del usuario y aumentar las ventas. Existen tres enfoques principales:

- **Filtrado colaborativo:** Recomienda ítems basándose en patrones de comportamiento similares entre usuarios o ítems.
- **Basado en contenido:** Utiliza características de los ítems y preferencias del usuario para generar recomendaciones.
- **Sistemas híbridos:** Combinan ambos enfoques para mejorar la precisión y relevancia.

Matrix Factorization y Embeddings

Una técnica fundamental en filtrado colaborativo es la **factorización de matrices**, que representa las interacciones usuario-ítem en espacios latentes. Dos métodos populares son:

- **SVD (Singular Value Decomposition):** Descompone la matriz de interacciones en factores latentes que capturan características implícitas.
- **ALS (Alternating Least Squares):** Optimiza iterativamente la factorización para grandes conjuntos de datos, eficiente para sistemas a escala.

Estas técnicas generan **embeddings** que permiten predecir la afinidad entre usuarios e ítems no observados.

***Ejemplo:** En Amazon, la matriz usuario-producto es enorme y dispersa. ALS permite factorizar esta matriz para descubrir patrones latentes y recomendar productos relevantes.*

¿Debería usar la Factorización de Matrices para sistemas de recomendación?



Made with  Napkin

Impacto en el Negocio

Los recomendadores influyen en KPIs como incremento de ventas, retención y satisfacción del cliente. La medición del impacto se realiza mediante:

- **Experimentación A/B:** Comparar grupos con y sin recomendaciones para evaluar uplift.
- **Métricas de negocio:** ROI, tasa de conversión, valor promedio de pedido.

Despliegue y Consideraciones Técnicas

Para operar en tiempo real, los sistemas deben:

- Minimizar latencia en generación de recomendaciones.
- Actualizar modelos periódicamente para reflejar cambios en preferencias.
- Integrar pipelines de scoring batch y streaming.

La generación de reportes ejecutivos reproducibles es esencial para comunicar resultados y decisiones a stakeholders. **Conocimiento clave:** La combinación de filtrado colaborativo y basado en contenido (sistemas híbridos) suele ofrecer mejores resultados en comercio electrónico, equilibrando precisión y diversidad (Ricci et al., 2011).

Contexto

Aplicación Práctica en Amazon

Amazon utiliza sistemas de recomendación híbridos que integran múltiples fuentes de información: historial de compras, navegación, valoraciones y características de productos. Estos sistemas permiten personalizar la experiencia de cada usuario, aumentando la probabilidad de compra y mejorando la satisfacción.

El pipeline típico incluye:

1. **Recolección y preprocesamiento de datos:** Extracción de interacciones usuario-producto y atributos relevantes.
2. **Feature engineering:** Creación de variables que capturan preferencias y contexto.
3. **Entrenamiento de modelos:** Uso de ALS para factorizar la matriz de interacciones y generar embeddings.
4. **Generación de recomendaciones:** Scoring de ítems candidatos en batch y actualización en streaming para eventos recientes.
5. **Evaluación y monitoreo:** Implementación de experimentos A/B para medir impacto y ajuste continuo.

Este enfoque permite a Amazon responder rápidamente a cambios en tendencias y comportamientos, manteniendo la relevancia y efectividad del sistema.

***Nota:** La integración de notebooks reproducibles facilita la colaboración entre equipos técnicos y de negocio, asegurando transparencia y trazabilidad en el análisis y despliegue.*

Práctica

Les pasamos código para que vean como funciona un sistema de recomendación de películas.

```
# Single Colab cell generated from the notebook: 'Sistema_Recomendador_tensorflow
(1).ipynb'
# All original markdown has been converted to commented lines starting with '#'.
# Code cells are included as-is.
# --- Cell 1 (markdown) ---
# ## Sistema de Recomendación de Películas
# Este cuaderno implementa un sistema básico de recomendación de películas
# utilizando técnicas de NLP y similitud de coseno.
# --- Cell 2 (markdown) ---
# ### Carga de Librerías y Datos Iniciales
# Esta celda importa las librerías necesarias para el análisis de datos.
# * **pandas (pd)**: carga el archivo CSV y trabaja con DataFrames.
# * **ast**: convierte strings tipo lista/dict en objetos Python con
ast.literal_eval.
# --- Cell 3 (code) ---
import pandas as pd
import ast
```

```

# URL de Google Drive
url = 'https://drive.google.com/uc?id=1UXMkOcO_gba8XvFEfha0prVxYUYVvPHx'
# Cargar el CSV
movies = pd.read_csv(url)
# Mostrar las primeras filas del dataset
movies.head()
# --- Cell 4 (markdown) ---
# ### Limpieza y Transformación de Columnas Complejas
# Muchas columnas contienen listas/dicts codificados como strings.
# Se usa ast.literal_eval para convertirlos a objetos Python reales.
# --- Cell 5 (code) ---
# Función para convertir columnas tipo lista/dict
def clean_column(col):
    return col.apply(lambda x: ast.literal_eval(x) if pd.notnull(x) else [])
# Limpiar algunas columnas del dataset
movies['genres'] = clean_column(movies['genres'])
movies['keywords'] = clean_column(movies['keywords'])
# Visualizar los cambios
movies[['genres', 'keywords']].head()
# --- Cell 6 (markdown) ---
# ### Conversión de Columnas con Listas de Diccionarios
# Se extraen únicamente los nombres de géneros y palabras clave.
# --- Cell 7 (code) ---
# Extraer solo el nombre de cada género y palabra clave
movies['genres'] = movies['genres'].apply(
    lambda x: [g['name'] for g in x] if isinstance(x, list) else [])
movies['keywords'] = movies['keywords'].apply(
    lambda x: [k['name'] for k in x] if isinstance(x, list) else [])
# Visualizar cambios
movies[['genres', 'keywords']].head()
# --- Cell 8 (markdown) ---
# ### Convertir Características en Texto
# Se combinan título, géneros y palabras clave en un único string por película
# para luego aplicar TF-IDF.
# --- Cell 9 (code) ---
# Crear una columna con texto combinado de características
movies['combined_features'] = movies.apply(
    lambda row: ' '.join(row['genres']) + ' ' + ' '.join(row['keywords']),
    axis=1)
movies[['title', 'combined_features']].head()
# --- Cell 10 (markdown) ---
# ### Vectorización del Texto con TF-IDF
# TF-IDF convierte texto en vectores numéricos que representan la importancia
# de cada palabra. Permite comparar películas por similitud de contenido.
# --- Cell 11 (code) ---
from sklearn.feature_extraction.text import TfidfVectorizer
# Inicializar TF-IDF
tfidf = TfidfVectorizer(stop_words='english')
# Ajustar y transformar
tfidf_matrix = tfidf.fit_transform(movies['combined_features'])
tfidf_matrix.shape
# --- Cell 12 (markdown) ---
# ### Cálculo de Similitud de Coseno
# Mide el ángulo entre dos vectores. Alta similitud = películas con géneros
# y keywords parecidos.
# --- Cell 13 (code) ---
from sklearn.metrics.pairwise import cosine_similarity
# Calcular la similitud de coseno entre todas las películas
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)

```



```

cosine_sim.shape
# --- Cell 14 (markdown) ---
# ### Construcción del Índice de Búsqueda
# Índice inverso: asigna cada título a su número de fila para búsqueda rápida.
# --- Cell 15 (code) ---
# Resetear índices para alinearlos correctamente
movies = movies.reset_index()
indices = pd.Series(movies.index, index=movies['title']).drop_duplicates()
indices.head()
# --- Cell 16 (markdown) ---
# ### Función de Recomendación
# get_recommendations(title):
# 1. Busca el índice de la película consultada.
# 2. Obtiene las similitudes con todas las demás.
# 3. Ordena por similitud descendente.
# 4. Devuelve las 10 películas más similares.
# --- Cell 17 (code) ---
def get_recommendations(title, cosine_sim=cosine_sim):
    # Verificar si existe el título
    if title not in indices:
        return ["La película no existe en la base de datos."]

    # Obtener el índice
    idx = indices[title]

    # Obtener los pares (índice, similitud)
    sim_scores = list(enumerate(cosine_sim[idx]))

    # Ordenar por similitud descendente
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)

    # Las 10 más similares (excluir la misma película)
    sim_scores = sim_scores[1:11]

    # Obtener los índices finales y devolver los títulos
    movie_indices = [i[0] for i in sim_scores]
    return movies['title'].iloc[movie_indices].tolist()
# --- Cell 18 (markdown) ---
# ### Ejemplo de Uso de la Función
# Se usa get_recommendations(title) para obtener recomendaciones personalizadas.
# --- Cell 19 (markdown) ---
# Dos pruebas de la función:
# * Recomendaciones para Toy Story
# * Recomendaciones para Jumanji
# --- Cell 20 (markdown) ---
# ### Pruebas de la Función de Recomendación
# Se demuestra el uso de get_recommendations y el comportamiento
# cuando el título no existe en el dataset.
# --- Cell 21 (code) ---
# Probamos:
print("Recomendaciones para Toy Story:")
print(get_recommendations('Toy Story'))
print("\nRecomendaciones para Jumanji:")
print(get_recommendations('Jumanji'))

```

Mazda – Segmentación de Clientes con Clustering

Introducción

¿Cómo puede una empresa como Mazda entender mejor a sus clientes para ofrecer productos y servicios personalizados que aumenten la satisfacción y las ventas? En esta unidad, abordaremos este desafío mediante la segmentación de clientes usando técnicas de clustering, un método de aprendizaje no supervisado que agrupa clientes con características similares. A partir de un conjunto de más de 30 variables, aprenderás a preparar los datos, seleccionar las características relevantes y aplicar algoritmos como K-Means y Gaussian Mixture Models (GMM) para identificar segmentos significativos. Además, exploraremos cómo traducir estos segmentos en estrategias de marketing concretas que impacten positivamente en el negocio. Este análisis práctico te permitirá comprender el pipeline completo de datos y modelado, y te preparará para replicar este tipo de soluciones en contextos reales de Data Science.

Teoría

Fundamentos del Clustering para Segmentación de Clientes

El **clustering** es una técnica de aprendizaje no supervisado que permite agrupar observaciones en segmentos homogéneos basados en sus características. En el contexto de Mazda, el objetivo es identificar grupos de clientes con comportamientos y atributos similares para diseñar estrategias de marketing personalizadas.

Conceptos Clave

- **Clustering:** Agrupamiento de datos sin etiquetas previas, buscando patrones latentes.
- **K-Means:** Algoritmo que particiona los datos en k clusters minimizando la varianza intra-cluster.
- **Gaussian Mixture Models (GMM):** Modelo probabilístico que asume que los datos provienen de una mezcla de distribuciones gaussianas, permitiendo clusters con formas elípticas.

Según Aggarwal (2015), el clustering es efectivo para segmentación cuando existen patrones latentes en atributos de clientes, facilitando la personalización y optimización de campañas.

Feature Engineering y Preparación de Datos

Antes de aplicar clustering, es fundamental preparar los datos:

- **Limpieza:** manejo de valores faltantes, corrección de errores.
- **Selección de variables:** elegir atributos relevantes que aporten información significativa.
- **Escalado:** normalizar variables para evitar sesgos por diferentes escalas.

Herramientas como `pandas` y `numpy` en Python facilitan estas tareas.

Pipeline de Datos para Clustering

Un pipeline típico incluye:

1. Ingestión y limpieza de datos
2. Selección y transformación de features
3. Escalado y normalización
4. Aplicación del algoritmo de clustering
5. Evaluación y validación de clusters
6. Interpretación y aplicación de resultados

Este flujo asegura reproducibilidad y calidad en el análisis.

Evaluación de Clusters

Para validar la calidad y estabilidad de los clusters se utilizan métricas como:

- **Inercia** (suma de distancias cuadradas dentro de clusters)
- **Silhouette Score** (medida de separación y cohesión)

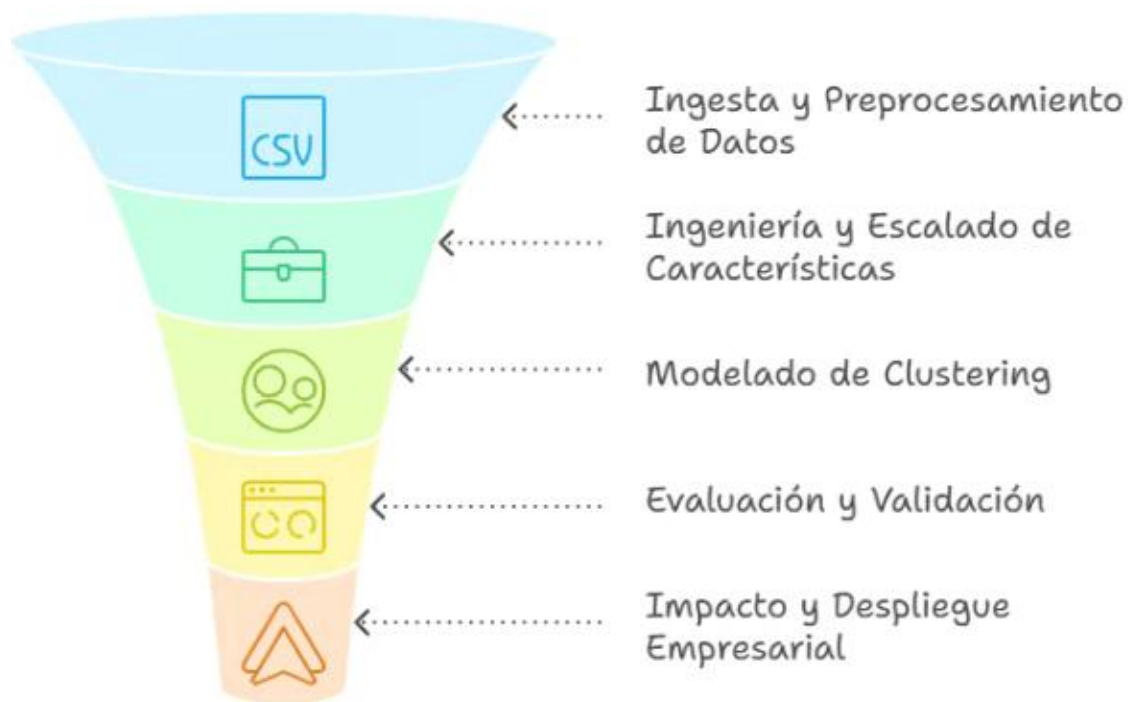
Estas métricas ayudan a decidir el número óptimo de segmentos y la robustez del modelo.

Medición del Impacto de Negocio

Traducir los resultados técnicos a indicadores de negocio es clave. Por ejemplo:

Métrica Técnica	Indicador de Negocio
Segmentos definidos y estables	Estrategias de marketing dirigidas y efectivas
Características distintivas	Personalización de ofertas y comunicación

Este enfoque conecta el análisis con el retorno de inversión y mejora continua.



3 - Teoría práctica: supervisado vs no supervisado

Este contenido ofrece un resumen práctico y aplicado de los algoritmos de aprendizaje supervisado y no supervisado, sus supuestos, ventajas y limitaciones, para que el estudiante pueda distinguir cuándo usar clustering, clasificación o reducción de dimensionalidad en problemas reales de ciencia de datos.

Introducción

Imagina que trabajas en una empresa de comercio electrónico y tienes un enorme conjunto de datos de clientes, pero no sabes qué patrones o grupos existen dentro de ellos. ¿Cómo podrías segmentar a tus clientes para campañas de marketing personalizadas? ¿O cómo predecir si un cliente realizará una compra en base a su comportamiento previo? Estas preguntas ilustran dos grandes enfoques en ciencia de datos: el aprendizaje supervisado y el no supervisado.

En esta unidad, reforzaremos tu comprensión del aprendizaje supervisado y te introduciremos al aprendizaje no supervisado, explorando sus algoritmos, supuestos, ventajas y limitaciones. Al finalizar, podrás distinguir claramente cuándo aplicar técnicas como clasificación, clustering o reducción de

dimensionalidad en problemas reales, una habilidad clave para diseñar soluciones efectivas en la industria de la ciencia de datos.

Aprendizaje Supervisado (REINFORCE)

El **aprendizaje supervisado** es un enfoque donde los modelos aprenden a partir de datos etiquetados, es decir, pares de entrada y salida conocidos. El objetivo es que el modelo generalice para predecir la salida correcta para nuevas entradas.

Tipos principales:

- **Clasificación:** Predice una categoría o clase. Ejemplo: determinar si un correo es spam o no.
- **Regresión:** Predice un valor numérico continuo. Ejemplo: estimar el precio de una vivienda.

Algoritmos comunes:

Algoritmo	Descripción breve	Ejemplo de uso en industria
Regresión lineal	Modela la relación lineal entre variables	Predicción de ventas según inversión publicitaria
Random Forest	Conjunto de árboles de decisión para clasificación o regresión	Detección de fraude en transacciones bancarias

Supuestos y consideraciones:

- Requiere datos etiquetados, lo que puede ser costoso o difícil de obtener.
- El modelo aprende patrones explícitos entre entrada y salida.
- Es fundamental elegir métricas adecuadas para evaluar el desempeño (precisión, recall, RMSE, etc.).

Aprendizaje No Supervisado (INTRODUCE)

El **aprendizaje no supervisado** trabaja con datos sin etiquetas, buscando estructuras o patrones ocultos.

Técnicas principales:

- **Clustering:** Agrupa datos similares en clusters o segmentos. Ejemplo: segmentación de clientes para marketing.
- **Reducción de dimensionalidad:** Simplifica datos complejos conservando la mayor información posible. Ejemplo: PCA (Análisis de Componentes Principales).

Algoritmos comunes:

Algoritmo	Descripción breve	Ejemplo de uso en industria
K-Means	Agrupa datos en k clusters basados en distancia	Segmentación de usuarios en plataformas digitales
PCA	Transforma variables correlacionadas en componentes independientes	Visualización y reducción de variables en análisis financiero

Supuestos y consideraciones:

- No requiere etiquetas, ideal para exploración de datos.
- Los resultados pueden ser menos interpretables que en supervisado.
- Es importante validar la calidad de los clusters o componentes obtenidos.

Comparación práctica

Aspecto	Aprendizaje Supervisado	Aprendizaje No Supervisado
Datos	Etiquetados (entrada-salida)	Sin etiquetas
Objetivo	Predecir o clasificar	Encontrar estructura o patrones
Ejemplos de aplicación	Detección de fraude, clasificación de imágenes	Segmentación de clientes, reducción de variables

***Nota:** La elección entre supervisado y no supervisado depende del problema, disponibilidad de datos y objetivo final.*

Aplicaciones en la Industria

En la práctica, los científicos de datos combinan ambos enfoques para resolver problemas complejos:

- **Segmentación de clientes:** Usando clustering para identificar grupos con comportamientos similares y luego aplicar modelos supervisados para predecir la respuesta a campañas.
- **Detección de fraude:** Modelos supervisados como random forest para clasificar transacciones sospechosas, apoyados por análisis no supervisados para descubrir patrones nuevos.
- **Optimización logística:** Reducción de dimensionalidad para simplificar variables y mejorar la eficiencia de modelos predictivos.

Este conocimiento te permitirá diseñar pipelines end-to-end que integren exploración, modelado y evaluación, adaptados a las necesidades reales del negocio.

4 - Métricas y estrategias de validación

En esta unidad, reforzaremos el conocimiento sobre métricas clave para evaluar modelos de clasificación, regresión y clustering, y exploraremos estrategias de validación robustas como k-fold y time-split.

Aprenderás a seleccionar las métricas y métodos de validación más adecuados según el problema de negocio, entendiendo sus ventajas, limitaciones y trade-offs para garantizar evaluaciones confiables y aplicables en entornos reales de ciencia de datos.

¿Cómo sabemos si un modelo de machine learning es realmente bueno?

Imagina que trabajas en una empresa de logística que quiere optimizar rutas de entrega usando modelos predictivos. ¿Cómo puedes estar seguro de que el modelo que desarrollaste realmente mejora la eficiencia y no solo funciona bien con los datos que ya tienes? La respuesta está en **evaluar correctamente el modelo** usando métricas adecuadas y estrategias de validación robustas.

En esta unidad, reforzaremos conceptos clave sobre métricas de evaluación para clasificación, regresión y clustering, y exploraremos métodos de validación como k-fold y time-split. Aprenderás a elegir la métrica y estrategia de validación más adecuada según el problema, entendiendo sus ventajas y limitaciones. Esto te permitirá tomar decisiones informadas y comunicar resultados con confianza en contextos reales de negocio.

Al finalizar, podrás seleccionar métricas y técnicas de validación que aseguren evaluaciones sólidas y aplicables, un paso fundamental para construir modelos confiables y útiles en la industria de ciencia de datos.

Métricas de evaluación: un repaso reforzado

Para evaluar modelos, es fundamental elegir métricas que reflejen el objetivo del negocio y la naturaleza del problema. A continuación, repasamos las métricas más comunes para clasificación, regresión y clustering, con ejemplos prácticos.

Clasificación

- **Accuracy (Precisión global):** proporción de predicciones correctas sobre el total. Útil cuando las clases están balanceadas.
- **Precision (Precisión):** proporción de verdaderos positivos sobre todos los positivos predichos. Importante cuando el costo de falsos positivos es alto.
- **Recall (Sensibilidad):** proporción de verdaderos positivos sobre todos los positivos reales. Clave cuando es crítico detectar todos los casos positivos.
- **F1-Score:** media armónica entre precision y recall, balancea ambos aspectos.
- **AUC-ROC:** área bajo la curva ROC, mide la capacidad del modelo para distinguir clases en diferentes umbrales.

***Ejemplo:** En detección de fraude, un alto recall es vital para no dejar pasar fraudes, aunque se sacrifiquen algunos falsos positivos.*

Regresión

- **RMSE (Root Mean Squared Error):** raíz del promedio de los errores al cuadrado, penaliza errores grandes.
- **MAE (Mean Absolute Error):** promedio de errores absolutos, más robusto a outliers.
- **R² (Coeficiente de determinación):** proporción de varianza explicada por el modelo.

***Ejemplo:** Para predecir demanda de productos, RMSE ayuda a entender el error típico en unidades vendidas.*

Clustering

- **Silhouette Score:** mide qué tan bien separado está cada cluster.
- **Davies-Bouldin Index:** evalúa la separación y compacidad de clusters.
- **Inercia:** suma de distancias cuadradas dentro de clusters, usada en k-means.

***Ejemplo:** En segmentación de clientes, un buen silhouette indica grupos bien definidos para campañas personalizadas.*

Estrategias de validación para evaluar modelos

La validación es clave para estimar el desempeño real del modelo en datos no vistos.

Validación simple (hold-out)

Dividir el dataset en entrenamiento y prueba. Rápido pero puede ser inestable si los datos son pocos.

Validación cruzada k-fold

- Se divide el dataset en k partes (folds).
- Se entrena el modelo k veces, cada vez con un fold diferente como prueba y el resto para entrenamiento.
- Se promedian las métricas para obtener una estimación robusta.

***Ventaja:** reduce la varianza en la estimación del desempeño.*

Validación temporal (time-split)

- Para datos secuenciales o series temporales, se respeta el orden temporal.
- Se entrena con datos anteriores y se prueba con datos posteriores.

***Ejemplo:** En predicción de demanda diaria, no se debe usar datos futuros para entrenar.*

Trade-offs en la selección de métricas y validación

- **Complejidad vs. interpretabilidad:** métricas simples como accuracy son fáciles de entender, pero pueden ser engañosas en datasets desbalanceados.
- **Tiempo de cómputo:** validación k-fold es más precisa pero consume más recursos.

- **Naturaleza del problema:** en problemas críticos (fraude, salud), priorizar recall o precisión según el impacto.
- **Datos disponibles:** en series temporales, usar time-split para evitar fugas de información.

***Reflexión:** No existe una métrica o estrategia universal; la elección debe alinearse con el contexto y objetivos del negocio.*

Aplicación práctica en la industria

En proyectos reales de ciencia de datos, la correcta elección de métricas y validación es decisiva para el éxito.

- **Detección de fraude:** se prioriza recall para minimizar falsos negativos, usando validación temporal para respetar la secuencia de transacciones.
- **Segmentación de clientes:** se evalúan métricas de clustering para definir grupos útiles para marketing personalizado.
- **Optimización logística:** se usan métricas de regresión para predecir tiempos de entrega, validando con k-fold para robustez.

Además, comunicar claramente estas elecciones a stakeholders facilita la toma de decisiones y la confianza en los modelos.

Esta unidad te prepara para seleccionar y justificar métricas y estrategias de validación que reflejen fielmente el desempeño real, un paso clave antes de avanzar a la interpretabilidad y despliegue de modelos.

Para practicar

Instrucciones:

```
# =====
#     MÉTRICAS Y ESTRATEGIAS DE VALIDACIÓN
# =====
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification, make_regression, make_blobs
from sklearn.model_selection import train_test_split, KFold, TimeSeriesSplit
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, mean_absolute_error, mean_squared_error,
    r2_score, silhouette_score, davies_bouldin_score)
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```

# =====
#                               PARTE 1 – CLASIFICACIÓN
# =====
print("\n=====")
print("    MÉTRICAS: CLASIFICACIÓN")
print("=====\\n")
# Dataset sintético de clasificación
Xc, yc = make_classification(
    n_samples=800, n_features=5, n_informative=3, n_redundant=0,
    weights=[0.7, 0.3], random_state=42)
# Hold-out
Xc_train, Xc_test, yc_train, yc_test = train_test_split(
    Xc, yc, test_size=0.3, random_state=42)
model_c = LogisticRegression()
model_c.fit(Xc_train, yc_train)
pred_c = model_c.predict(Xc_test)
proba_c = model_c.predict_proba(Xc_test)[: ,1]
print("Accuracy :", accuracy_score(yc_test, pred_c))
print("Precision:", precision_score(yc_test, pred_c))
print("Recall   :", recall_score(yc_test, pred_c))
print("F1-score :", f1_score(yc_test, pred_c))
print("AUC-ROC  :", roc_auc_score(yc_test, proba_c))
# =====
#                               PARTE 2 – REGRESIÓN
# =====
print("\n=====")
print("    MÉTRICAS: REGRESIÓN")
print("=====\\n")
Xr, yr = make_regression(
    n_samples=800, n_features=4, noise=20, random_state=42)
Xr_train, Xr_test, yr_train, yr_test = train_test_split(
    Xr, yr, test_size=0.3, random_state=42)
model_r = LinearRegression()
model_r.fit(Xr_train, yr_train)
pred_r = model_r.predict(Xr_test)
print("MAE :", mean_absolute_error(yr_test, pred_r))
print("RMSE:", np.sqrt(mean_squared_error(yr_test, pred_r)))
print("R²  :", r2_score(yr_test, pred_r))
# =====
#                               PARTE 3 – CLUSTERING
# =====
print("\n=====")
print("    MÉTRICAS: CLUSTERING")
print("=====\\n")
Xcl, ycl = make_blobs(
    n_samples=600, centers=3, cluster_std=1.0, random_state=42)
scaler = StandardScaler()
Xcl_scaled = scaler.fit_transform(Xcl)
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(Xcl_scaled)
print("Silhouette Score      :", silhouette_score(Xcl_scaled, labels))
print("Davies-Bouldin Index :", davies_bouldin_score(Xcl_scaled, labels))
print("Inertia (k-means)     :", kmeans.inertia_)
plt.scatter(Xcl[:,0], Xcl[:,1], c=labels, cmap="viridis")
plt.title("Clustering - KMeans")
plt.show()

```

```

# =====
#                               PARTE 4 – VALIDACIÓN K-FOLD
# =====
print("\n=====")
print("  ESTRATEGIAS: K-FOLD")
print("=====\\n")
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = []
for train_i, test_i in kf.split(Xc):
    model = LogisticRegression()
    model.fit(Xc[train_i], yc[train_i])
    pred = model.predict(Xc[test_i])
    scores.append(accuracy_score(yc[test_i], pred))
print("Scores por fold:", scores)
print("Promedio K-fold :", np.mean(scores))
# =====
#                               PARTE 5 – VALIDACIÓN TEMPORAL (TIME-SPLIT)
# =====
print("\n=====")
print("  VALIDACIÓN TEMPORAL (SERIES)")
print("=====\\n")
# Dataset simulado con orden temporal
n = 500
Xt = np.arange(n).reshape(-1,1)  # tiempo como feature
yt = np.sin(np.arange(n)/20) + np.random.normal(0, 0.2, n)
tscv = TimeSeriesSplit(n_splits=5)
fold_num = 1
for train_idx, test_idx in tscv.split(Xt):
    model = LinearRegression()
    model.fit(Xt[train_idx], yt[train_idx])
    pred = model.predict(Xt[test_idx])
    score = mean_squared_error(yt[test_idx], pred)
    print(f"Fold {fold_num} - MSE:", score)
    fold_num += 1
plt.plot(yt, label="serie real")
plt.title("Serie temporal simulada")
plt.legend()
plt.show()

```

[Atrás](#)